CURSO BACKEND JAVA & QUARKUS — AULA 03 (MANUAL DE CAMPO)

Anotações ("Tags") Essenciais no Quarkus, Java e Swagger

Em Java e Quarkus, o que comumente chamamos de "tags" são tecnicamente conhecidas como **Anotações (Annotations)**, identificadas pelo símbolo `@`. As anotações são metadados inseridos no código fonte que instruem o compilador e o framework sobre como processar, mapear e gerenciar cada classe, atributo ou método em tempo de execução e compilação.

1. MÓDULO DE PERSISTÊNCIA E ENTIDADES (`ENTITY / MODEL`)

As anotações desta camada instruem o ORM (Hibernate / Quarkus Hibernate ORM Panache) sobre como traduzir objetos Java em tabelas e colunas do banco de dados relacional.

Anotação / "Tag"	Origem / Pacote	O que faz (Explicação Simplificada)
`@Entity`	`jakarta.persistence`	Avisa ao framework: "Esta classe Java representa uma tabela no banco de dados".
`@Table`	`jakarta.persistence`	Define explicitamente o nome real da tabela no banco (ex: `name = "tb_produto"`).
`@Id`	`jakarta.persistence`	Marca o atributo como a **Chave Primária (Primary Key)** da tabela.
`@GeneratedValue`	`jakarta.persistence`	Delega ao banco a geração automática do ID (ex: autoincremento/sequence).
`@Column`	`jakarta.persistence`	Configura o nome da coluna, tamanho (`length`) e restrições (`nullable = false`).
`@Enumerated`	`jakarta.persistence`	Grava enums como texto (`EnumType.STRING`) em vez de números inteiros ordinais.
`@ManyToOne`	`jakarta.persistence`	Mapeia relacionamento de Chave Estrangeira (N instâncias desta pertencem a 1 de outra).

1.1. Exemplo Prático no Quarkus: Entidade com Hibernate ORM com Panache

No Quarkus, ao estender `PanacheEntity` ou `PanacheEntityBase`, ganhamos métodos estáticos de CRUD prontos (como `Produto.listAll()` ou `Produto.findById(id)`) sem precisar escrever interfaces de Repository manuais!

```
package com.empresa.domain;

import io.quarkus.hibernate.orm.panache.PanacheEntity;
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "tb_produto")
public class Produto extends PanacheEntity { // PanacheEntity já nos dá o atributo '@Id Long id'
automaticamente!

    @Column(name = "nome_produto", nullable = false, length = 100)
    public String nome;

    @Column(precision = 10, scale = 2, nullable = false)
    public BigDecimal preco;

    @Enumerated(EnumType.STRING)
    @Column(name = "status_prod")
    public StatusProduto status;
}
```

2. MÓDULO DE EXPOSIÇÃO REST (`RESOURCE / CONTROLLER`)

O Quarkus utiliza o padrão **Jakarta REST (JAX-RS)** via biblioteca RESTEasy Reactive para transformar métodos Java em endpoints HTTP de alta performance.

Anotação / "Tag"	Origem / Pacote	O que faz (Explicação Simplificada)
`@Path`	`jakarta.ws.rs`	Define a URI/rota base do endpoint (ex: `@Path("/api/v1/produtos")`).
`@GET`, `@POST`	`jakarta.ws.rs`	Associa o método Java ao verbo HTTP correspondente (Buscar dados ou Criar dados).
`@PUT`, `@DELETE`	`jakarta.ws.rs`	Associa o método ao verbo HTTP para Atualização total ou Remoção de recurso.
`@Produces`	`jakarta.ws.rs`	Define o formato da resposta devolvida ao cliente (geralmente `APPLICATION_JSON`).
`@Consumes`	`jakarta.ws.rs`	Define o formato do corpo da requisição que o método aceita (`APPLICATION_JSON`).
`@RestPath` / `@PathParam`	`org.jboss.resteasy`	Extrai variáveis da própria URL (ex: `/produtos/{id}` extrai o valor de `{id}`).
`@RestQuery` / `@QueryParam`	`org.jboss.resteasy`	Extrai parâmetros passados na query string da URL (ex: `/produtos?nome=notebook`).

2.1. Exemplo Prático: Resource REST em Quarkus

O recurso abaixo recebe chamadas HTTP, extrai parâmetros da URL e delega o processamento:

```
package com.empresa.resource;

import com.empresa.domain.Produto;
import org.jboss.resteasy.reactive.RestPath;
import org.jboss.resteasy.reactive.RestQuery;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import java.util.List;

@Path("/api/v1/produtos")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProdutoResource {

    @GET
    public List<Produto> listarTodos(@RestQuery String nome) {
        if (nome != null) {
            return Produto.find("nome like ?1", "%" + nome + "%").list();
        }
        return Produto.listAll();
    }

    @GET
    @Path("/{id}")
    public Response buscarPorId(@RestPath Long id) {
        Produto produto = Produto.findById(id);
        if (produto == null) {
            return Response.status(Response.Status.NOT_FOUND).build();
        }
        return Response.ok(produto).build();
    }
}
```

3. MÓDULO DE INJEÇÃO DE DEPENDÊNCIAS E SERVIÇOS (`SERVICE / CDI`)

3.1. Escopos e Gerenciamento de Componentes (CDI)

O Quarkus utiliza CDI (Contexts and Dependency Injection) com o motor ultra-rápido **Arc**. As anotações definem

3. MÓDULO CDI E SERVIÇOS (CONTINUAÇÃO)

3.2. Exemplo Prático: Serviço de Negócio Anotado

Abaixo vemos a camada de serviço desacoplada, utilizando injeção de dependência e controle transacional:

```
package com.empresa.service;

import com.empresa.domain.Produto;
import com.empresa.dto.ProdutoForm;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import jakarta.transaction.Transactional;

@ApplicationScoped // Define que esta classe é um Bean singleton gerenciado pelo Quarkus
public class ProdutoService {

    @Transactional // Garante que a gravação ocorra dentro de uma transação segura
    public Produto criarNovoProduto(ProdutoForm form) {
        Produto produto = new Produto();
        produto.nome = form.nome();
        produto.preco = form.preco();
        produto.persist(); // Salva no banco de dados via Panache
        return produto;
    }
}
```

4. MÓDULO DE VALIDAÇÃO DE ENTRADA (`BEAN VALIDATION / DTOS`)

As anotações do pacote `jakarta.validation.constraints` garantem que os dados enviados pelos clientes nas requisições sigam rigorosamente as regras do sistema antes mesmo de atingirem a lógica de negócios.

Anotação / "Tag"	Regra de Validação Aplicada	Uso Recomendado em DTOs
`@NotNull`	O valor não pode ser `null`. Aceita strings vazias `""`.	Objetos Wrapper, Datas, Enums, Numbers.
`@NotBlank`	Não pode ser null, nem vazia, e não pode conter apenas espaços.	**Campos de Texto obrigatórios** (Nomes, CPFs).
`@Size`	Valida o número mínimo e máximo de caracteres ou elementos.	Ex: `@Size(min = 3, max = 100)`.
`@Email`	Valida se o texto possui formato sintático válido de e-mail.	Campos de login e contato de usuários.
`@Min` / `@Max`	Garante um valor numérico mínimo e máximo permitido.	Ex: `@Min(0)` para impedir valores negativos.
`@Valid`	Avisa ao framework: "Execute também a validação dos objetos filhos".	Colocado em métodos de Controller antes do `@RequestBody` / parâmetro.

4.1. Exemplo Prático: Formulario DTO com Validação Robusta

Uso de um Java Record anotado com Bean Validation para servir de contrato seguro de entrada na API:

```
package com.empresa.dto;

import jakarta.validation.constraints.*;
import java.math.BigDecimal;

public record ProdutoForm(
    @NotBlank(message = "O nome do produto é obrigatório.")
    @Size(min = 2, max = 100, message = "O nome deve ter entre 2 e 100 caracteres.")
    String nome,

    @NotNull(message = "O preço é obrigatório.")
    @Positive(message = "O preço deve ser maior que zero.")
    BigDecimal preco
) {}
```

5. MÓDULO DE DOCUMENTAÇÃO OPENAPI / SWAGGER (`SMALLRYE OPENAPI`)

No Quarkus, a documentação interativa do Swagger é gerada utilizando anotações da especificação **MicroProfile OpenAPI** (pacote `org.eclipse.microprofile.openapi.annotations`).

Anotação / "Tag"	Finalidade na Documentação da API
`@Tag`	Agrupa endpoints no painel do Swagger por módulo (ex: `name = "Produtos"`).
`@Operation`	Descreve a ação do endpoint (`summary = "Descrição curta"`, `description = "Detalhada"`).
`@APIResponse`	Documenta os possíveis códigos de retorno HTTP (ex: 200 OK, 400 Bad Request, 404 Not Found).
`@Schema`	Personaliza a descrição, exemplo e visibilidade dos campos do DTO na interface Swagger.
`@Parameter`	Documenta individualmente um parâmetro recebido via URL, Header ou Query String.

5.1. Exemplo Integrado Completo: Quarkus + Bean Validation + Swagger

Código de um Resource profissional combinando exposição REST, validações e documentação OpenAPI:

```
package com.empresa.resource;

import com.empresa.dto.ProdutoForm;
import com.empresa.domain.Produto;
import jakarta.inject.Inject;
import jakarta.validation.Valid;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.*;
import org.eclipse.microprofile.openapi.annotations.Operation;
import org.eclipse.microprofile.openapi.annotations.media.Content;
import org.eclipse.microprofile.openapi.annotations.media.Schema;
import org.eclipse.microprofile.openapi.annotations.responses.APIResponse;
import org.eclipse.microprofile.openapi.annotations.tags.Tag;

@Path("/api/v1/produtos")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
@Tag(name = "Produtos", description = "Gestão do catálogo de produtos do sistema")
public class ProdutoController {

    @POST
    @Operation(summary = "Cadastra um novo produto", description = "Cria um registro de produto e
retorna o objeto criado com ID")
    @APIResponse(responseCode = "201", description = "Produto criado com sucesso",
                 content = @Content(schema = @Schema(implementation = Produto.class)))
    @APIResponse(responseCode = "400", description = "Dados de entrada inválidos ou inconsistentes")
    public Response cadastrar(@Valid ProdutoForm form) { // @Valid dispara o Bean Validation do DTO!
        Produto produto = new Produto();
        produto.nome = form.nome();
        produto.preco = form.preco();
        produto.persist();
        return Response.status(Response.Status.CREATED).entity(produto).build();
    }
}
```

5.2. Resumo de Importações Recomendadas por Pacote

- **JPA/Persistência:** `import jakarta.persistence.*;`
- **REST (Endpoints/Métodos):** `import jakarta.ws.rs.*;`
- **Injeção & Escopo (CDI):** `import jakarta.enterprise.context.*; import jakarta.inject.Inject;`
- **Validações:** `import jakarta.validation.constraints.*; import jakarta.validation.Valid;`
- **Swagger/OpenAPI:** `import org.eclipse.microprofile.openapi.annotations.*;`